

Jenkins task 3

Task 1

Create one Jenkins job using the below code and create three stages:

- **stage1**: Git clone to download the source code.
- **stage2**: Sonarqube Integration to check the quality of code.
- **stage3**: Slack Integration to send the alerts to slack.

URL: <https://github.com/betawins/VProfile-1.git>

- **stage1**: Git clone to download the source code.

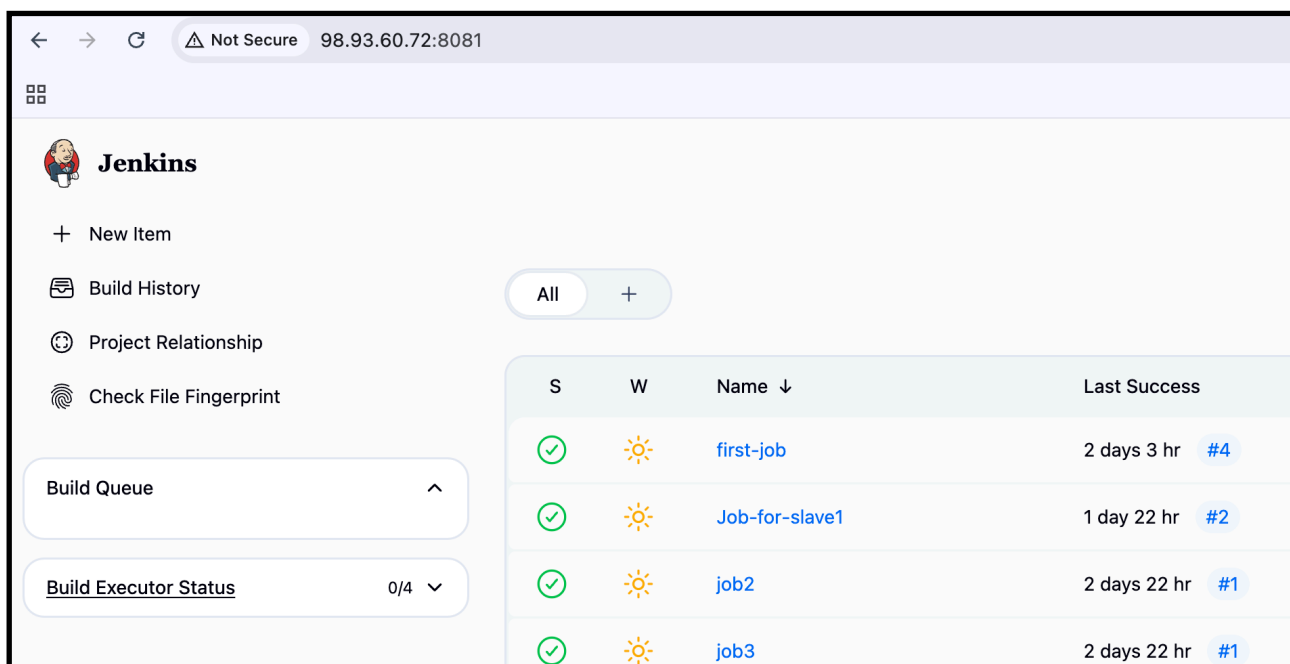
Follow the below steps

—> step 1 :

—> open your Jenkins dashboard

- Go to your Jenkins URL → `http://<your-server-ip>:8080/`

—> iam using 8081 as port number for my Jenkins application

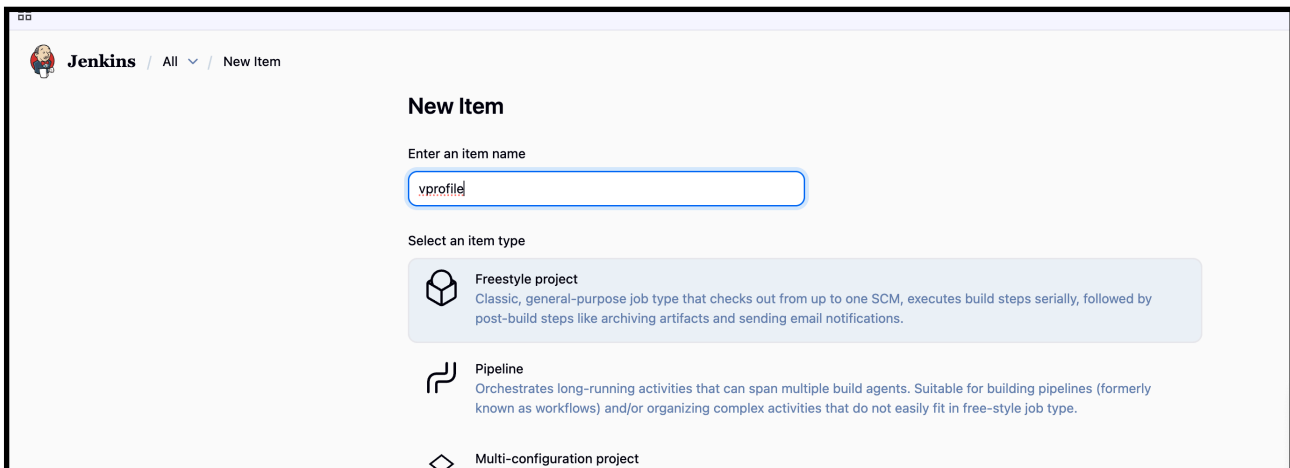


The screenshot shows the Jenkins dashboard in a web browser. The address bar indicates the URL is `http://98.93.60.72:8081`. The dashboard features a sidebar on the left with navigation links: "New Item", "Build History", "Project Relationship", and "Check File Fingerprint". Below these links are two status boxes: "Build Queue" (empty) and "Build Executor Status" (0/4). The main area displays a table of jobs with columns for status (S), warnings (W), name, and last success. All jobs shown are in a successful state (green checkmark).

S	W	Name ↓	Last Success
✓	☀	first-job	2 days 3 hr #4
✓	☀	Job-for-slave1	1 day 22 hr #2
✓	☀	job2	2 days 22 hr #1
✓	☀	job3	2 days 22 hr #1

Step 2: Create a New Job

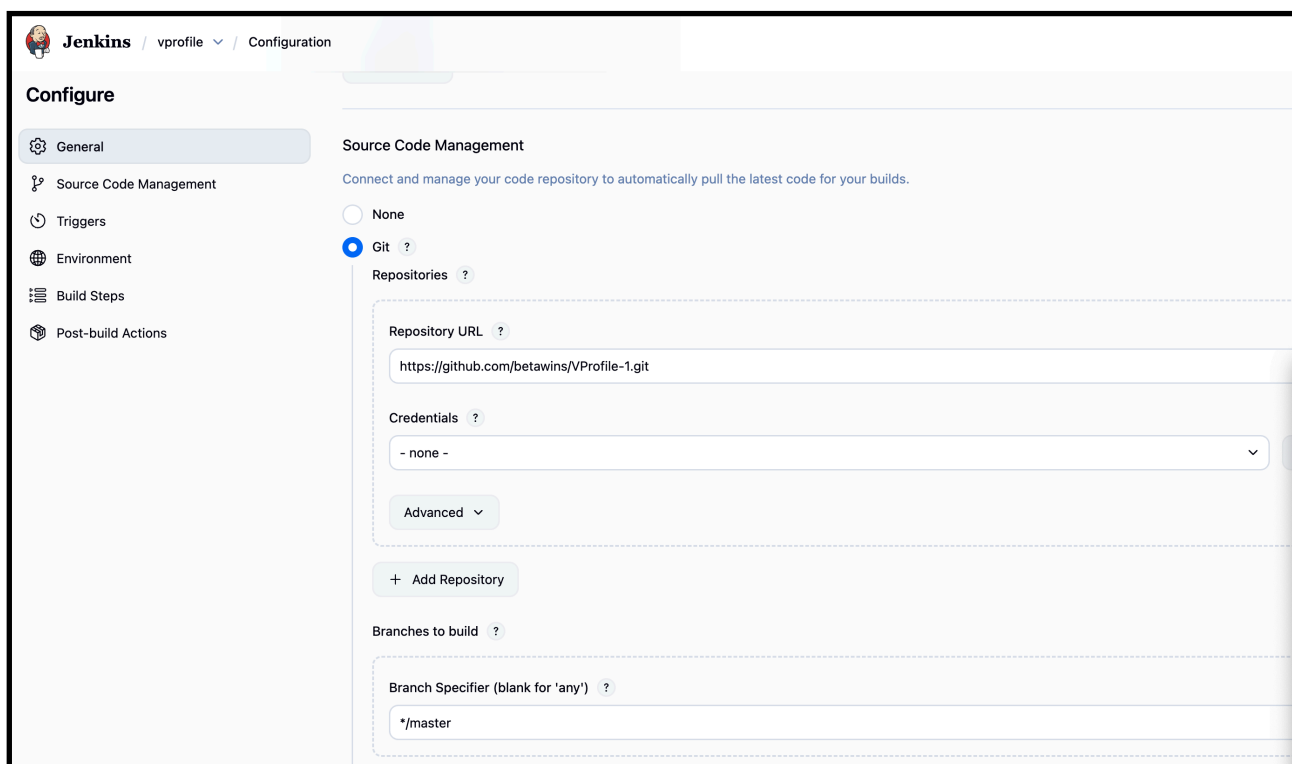
1. Click on “New Item”
2. Enter a job name → e.g. - VProfile.
3. Select “Freestyle project”.
4. Click **OK**.



Step 3: Configure Git Repository

1. In the job configuration page:
 - Scroll to **Source Code Management**.
 - Select **Git**.
 - In **Repository URL**, enter your Git repo link:

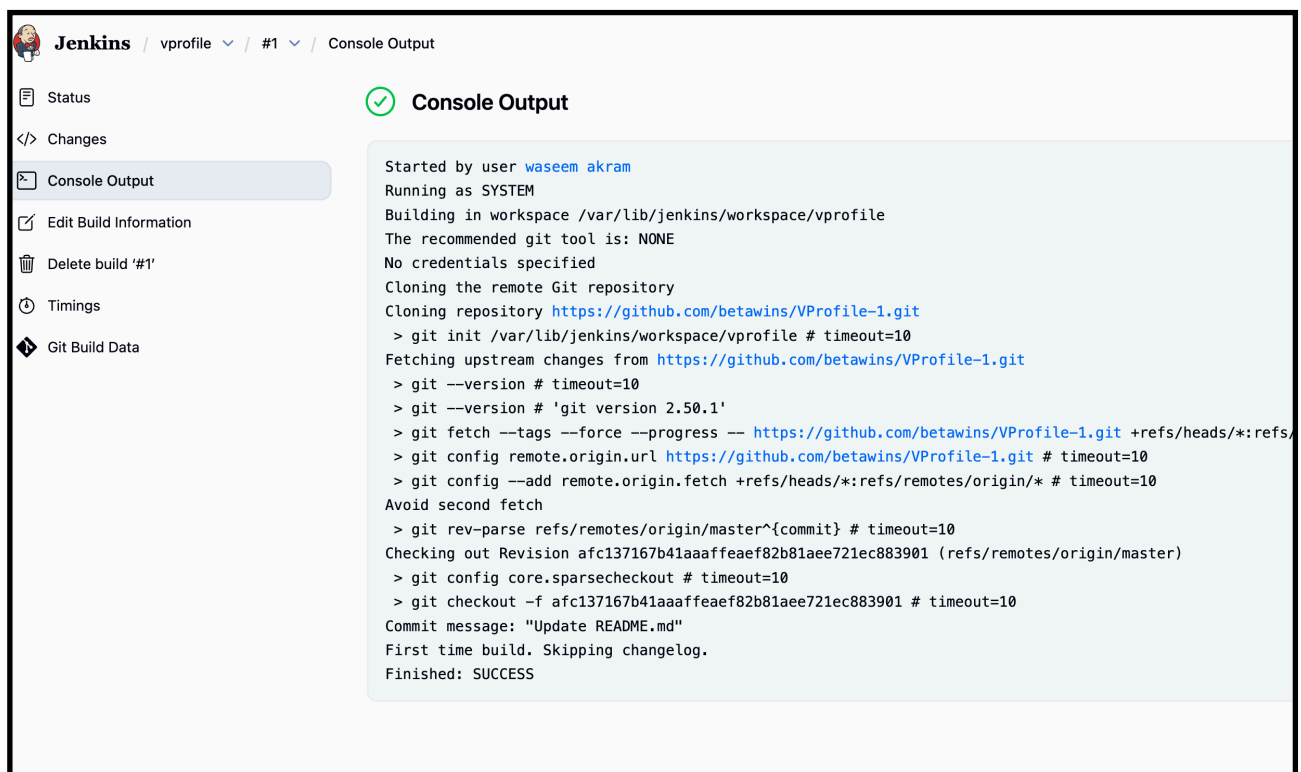
<https://github.com/betawins/VProfile-1.git>



In **Branches to build**, specify the branch (default: */main or */master).

Step 4: Save and Build

- Click **Save**.
- Click **Build Now**.
- Check the **Console Output** → It should show Jenkins cloning the repository.



The screenshot shows the Jenkins web interface for a build named 'vprofile' with ID '#1'. The 'Console Output' tab is selected, displaying a green checkmark icon and the title 'Console Output'. The output text shows the build process: it started by user 'waseem akram', running as 'SYSTEM', and building in the workspace '/var/lib/jenkins/workspace/vprofile'. It then clones the remote Git repository 'https://github.com/betawins/VProfile-1.git' using 'git init' and 'git fetch'. The output concludes with 'Commit message: "Update README.md"', 'First time build. Skipping changelog.', and 'Finished: SUCCESS'.

```
Started by user waseem akram
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/vprofile
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/betawins/VProfile-1.git
> git init /var/lib/jenkins/workspace/vprofile # timeout=10
Fetching upstream changes from https://github.com/betawins/VProfile-1.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/VProfile-1.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/betawins/VProfile-1.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision afc137167b41aaaffaeaf82b81aee721ec883901 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f afc137167b41aaaffaeaf82b81aee721ec883901 # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
Finished: SUCCESS
```

—->we have successfully **Git clone to download the source code.**

From url <https://github.com/betawins/VProfile-1.git>

—-> for checking the info of the cloned repo **We can check by login to your ssh**

cd /var/lib/Jenkins

cd workspace

```
[root@ip-172-31-65-24 workspace]# ls
first-job job2 job3 private private@tmp vprofile
[root@ip-172-31-65-24 workspace]# cd vprofile
[root@ip-172-31-65-24 vprofile]# ls
Dockerfile README.md ansible azure-pipelines.yml pom.xml src
[root@ip-172-31-65-24 vprofile]# ll
total 20
-rw-r--r--. 1 jenkins jenkins 137 Dec 26 11:24 Dockerfile
-rw-r--r--. 1 jenkins jenkins 978 Dec 26 11:24 README.md
drwxr-xr-x. 2 jenkins jenkins 91 Dec 26 11:24 ansible
-rw-r--r--. 1 jenkins jenkins 508 Dec 26 11:24 azure-pipelines.yml
-rw-r--r--. 1 jenkins jenkins 5830 Dec 26 11:24 pom.xml
drwxr-xr-x. 4 jenkins jenkins 30 Dec 26 11:24 src
[root@ip-172-31-65-24 vprofile]#
```

—> we are able to download the clone repo

—>stage2:

Sonarqube Integration to check the quality of code.

Prerequisites

- EC2 instance with Java installed
- Use t2.large with atleast 20gb memory to run sonarqube.

1.Step-by-Step SonarQube Installation on EC2

- Launch EC2 Instance
- Instance type: **t2.large**
- Storage: **20 GB** (minimum).

☒	jenkins-master	i-0fe26bc99f7f8361b	Running	t3.micro	3/3 checks passec View al
☐	sonarqube-1	i-0527d2952197c1bca	Running	t3.small	Initializing View al
☐	backup-jenkins	i-04e3c87221f06a1c7	Stopped	t3.micro	– View al

Step1 : install docker

Yum install -y docker

```
[ec2-user@ip-172-31-89-80 ~]$ sudo -i
[root@ip-172-31-89-80 ~]# yum install docker -y
Last metadata expiration check: 0:00:19 ago on Mon Dec 29 07:55:25 2025.
Dependencies resolved.
=====
Package                                                    Architecture
=====
Installing:
docker                                                    x86_64
Installing dependencies:
container-selinux                                         noarch
containerd                                                x86_64
iptables-libs                                             x86_64
iptables-nft                                              x86_64
libcgroup                                                 x86_64
libnetfilter_conntrack                                   x86_64
libnftnl                                                  x86_64
libnftnl                                                  x86_64
pigz                                                       x86_64
runc                                                       x86_64
=====
Transaction Summary
=====
Install 11 Packages

Total download size: 74 M
Installed size: 280 M
Downloading Packages:
(1/11): container-selinux-2.242.0-1.amzn2023.noarch.rpm
(2/11): iptables-libs-1.8.8-3.amzn2023.0.2.x86_64.rpm
(3/11): iptables-nft-1.8.8-3.amzn2023.0.2.x86_64.rpm
(4/11): libcgroup-3.0-1.amzn2023.0.1.x86_64.rpm
(5/11): libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64.rpm
(6/11): libnftnl-1.0.1-19.amzn2023.0.2.x86_64.rpm
(7/11): libnftnl-1.2.2-2.amzn2023.0.2.x86_64.rpm
(8/11): pigz-2.5-1.amzn2023.0.3.x86_64.rpm
```

--> systemctl start docker

--> docker pull sonarqube

```
Complete!
[root@ip-172-31-89-80 ~]# systemctl start docker
[root@ip-172-31-89-80 ~]# docker pull sonarqube
Using default tag: latest
latest: Pulling from library/sonarqube
20043066d3d5: Pull complete
f878172f1ac3: Pull complete
0b92fa77023f: Pull complete
1321ce5b3e3a: Pull complete
4a0e444f3c26: Pull complete
f60cf9719a3e: Downloading [=====] 436.6MB/901.5MB
2a0225763c5a: Download complete
4f4fb700ef54: Download complete
```

—-> docker image

```
[root@ip-172-31-89-80 ~]# docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
sonarqube     latest   247ce055a80b   2 weeks ago   1.27GB
[root@ip-172-31-89-80 ~]#
```

—-> docker run -itd -p 9000:9000 sonarqube

```
sonarqube     latest   247ce055a80b   2 weeks ago   1.27GB
[root@ip-172-31-89-80 ~]# docker run -itd -p 9000:9000 sonarqube
a164990fb54f07b09e6691d1d7bdaa9a8aad789af5641325ba31d43f3ba84b6b
[root@ip-172-31-89-80 ~]#
```

—-> docker ps

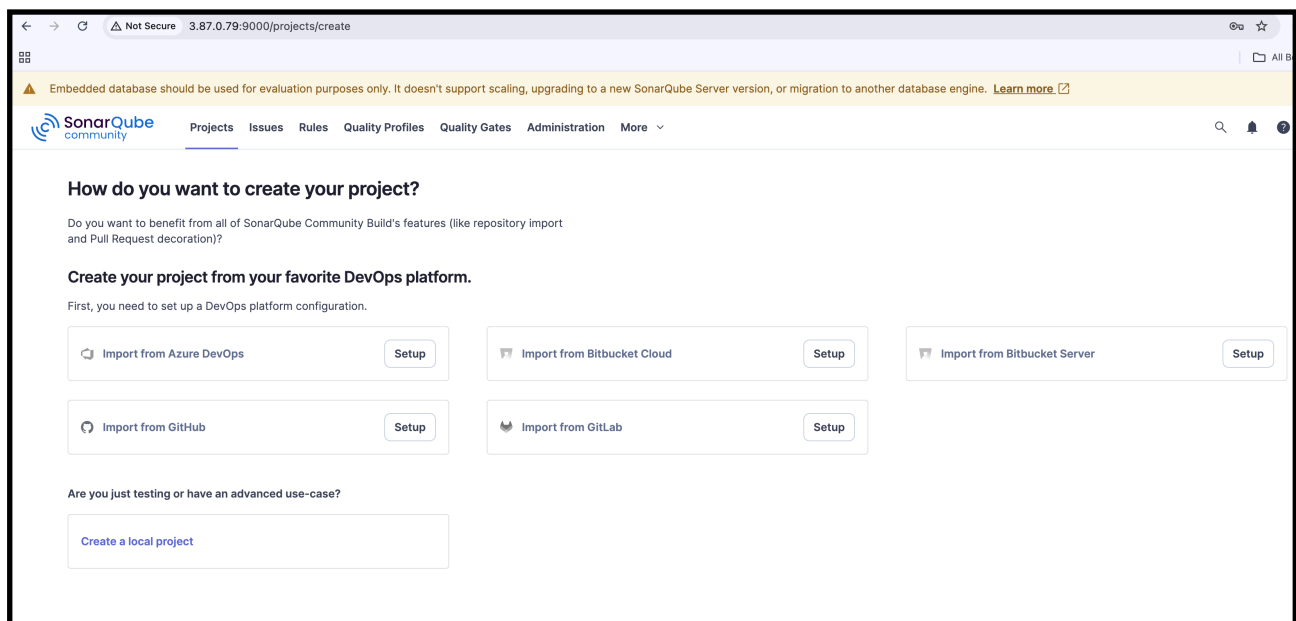
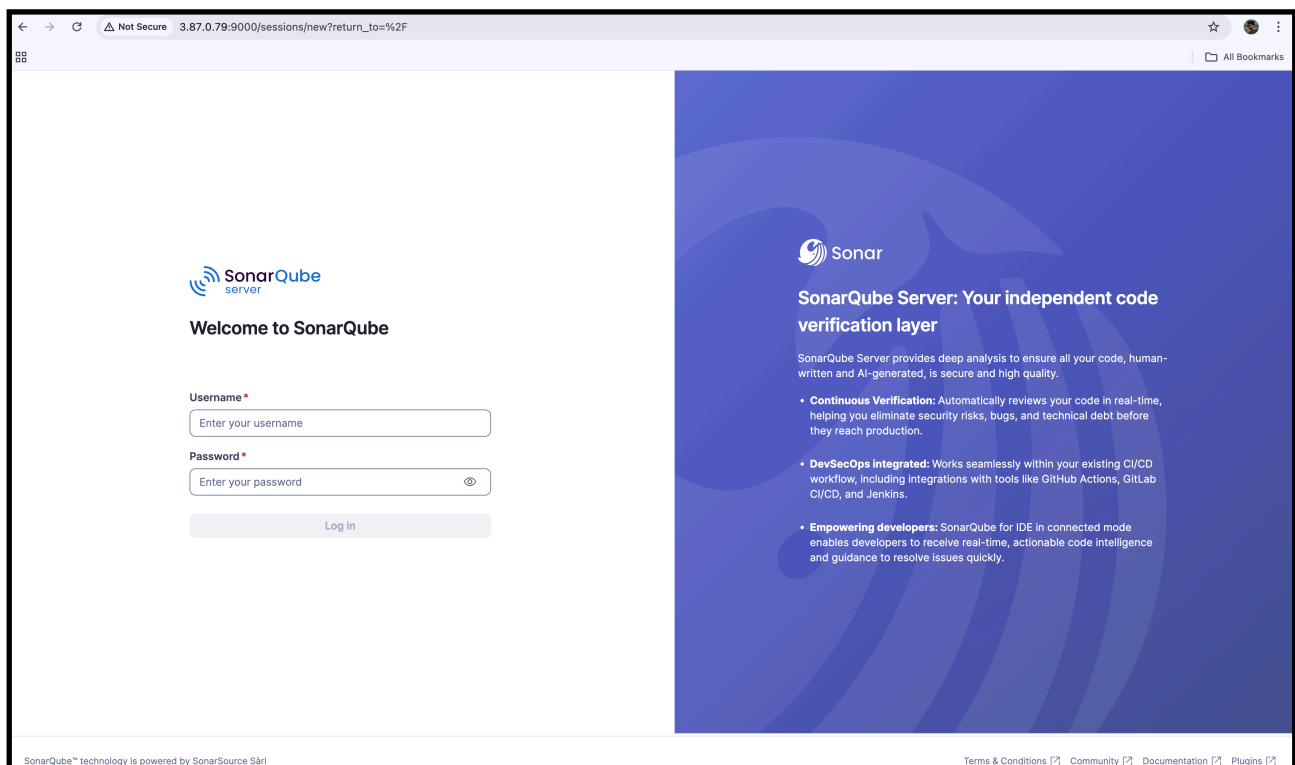
```
a164990fb54f07b09e6691d1d7bdaa9a8aad789af5641325ba31d43f3ba84b6b
[root@ip-172-31-89-80 ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
a164990fb54f   sonarqube "/opt/sonarqube/dock_    27 seconds ago Up 25 seconds 0.0.0.0:9000->9000/tcp, :::9000->9000/tcp flamboyant_fermat
[root@ip-172-31-89-80 ~]#
```

Step 2:

—> now login to your sonarqube

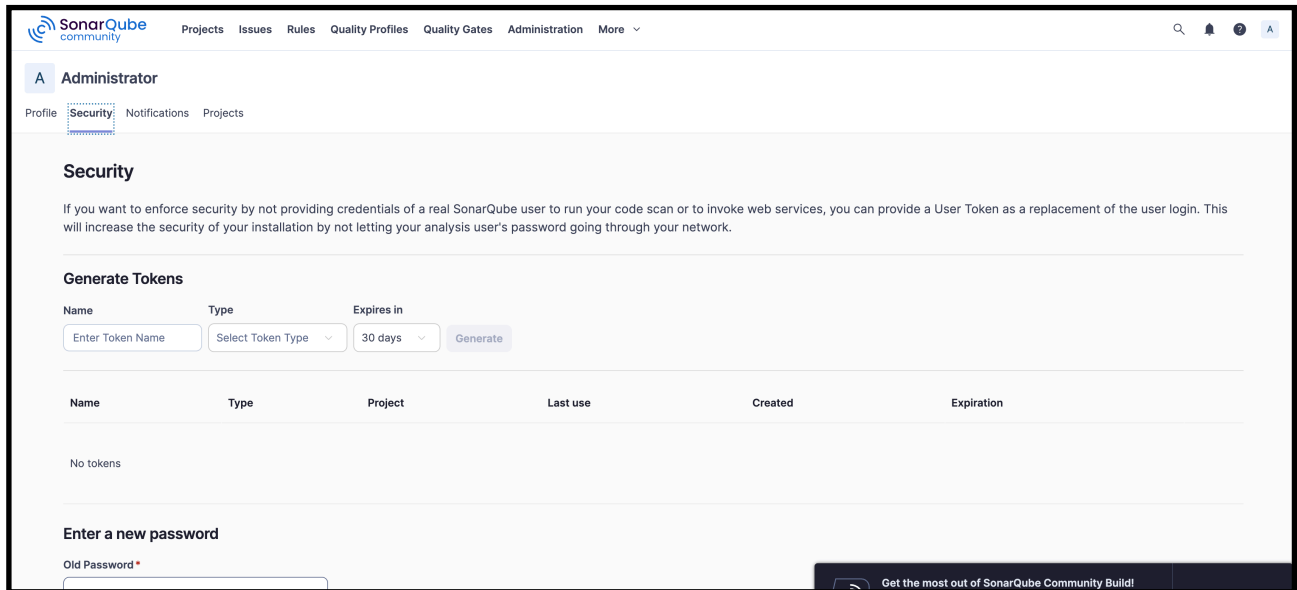
—-> add the id password

—-> and later give your own password and save

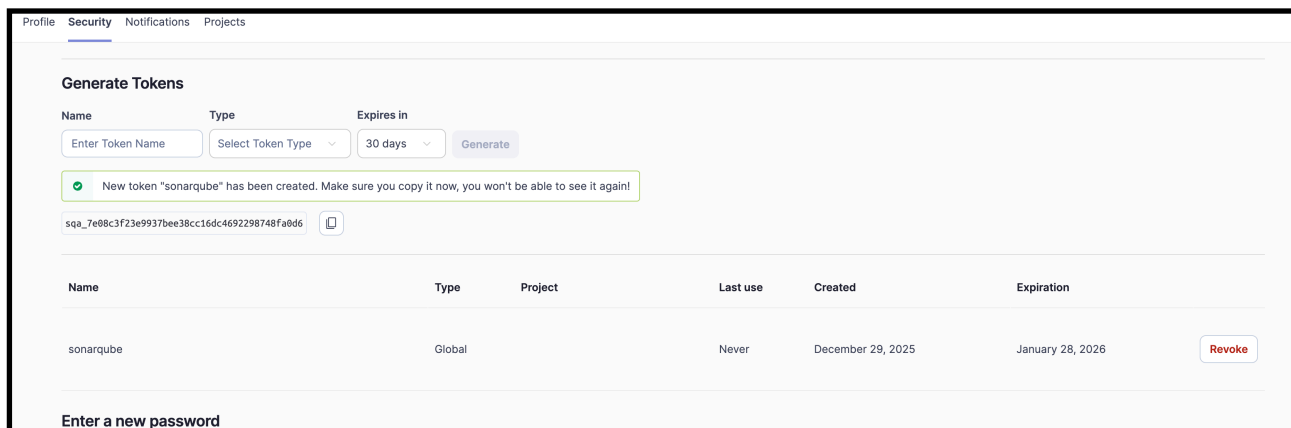


—-> after login go to the security

—-> and generate a token



—-> copy the token



Step 3:

- > now go to your Jenkins master
- > create new credentials
- > select the secret text
 - > in side secret paste the token from sonarqube
- > save

New credentials

Kind
Secret text

Scope ?
Global (Jenkins, nodes, items, all child items, etc)

Secret
.....

ID ?
sonar-token

Description ?
sonar-token

Create

—-> after creating the credential

—> go to Jenkins —> manage Jenkins —> plugins —> available plugins
 —> search sonarqube scanner
 —> and install it

Plugins

sonarqube

Install	Name ↓	Released	Health
☒	SonarQube Scanner 2.18.2 External Site/Tool Integrations Build Reports This plugin allows an easy integration of SonarQube , the open source platform for Continuous Inspection of code quality.	18 days ago	84
☐	Sonar Gerrit 388.v9b_f1cb_e42306 External Site/Tool Integrations This plugin allows to submit issues from SonarQube to Gerrit as comments directly.	1 yr 6 mo ago	86
☐	SonarQube Generic Coverage 1.0 TODO	6 yr 5 mo ago	76

—-> now go to manage Jenkins —> roll

STEP 4 — Configure SonarQube Server in Jenkins

Manage Jenkins

➔ **System**

➔ Scroll to **SonarQube** servers

➔ **Add SonarQube**

Fill:

SonarQube installations

List of SonarQube installations

Name

sonarqube-server

This property is mandatory.

Server URL

Default is `http://localhost:9000`

`http://52.87.178.31:9000`

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

sonar-token

Advanced

+ Add

+ Add SonarQube

Field	Value
Name	SonarQube
Server URL	<code>http://<YOUR-INSTANCE-PUBLIC-IP>:9000</code>
Authentication Token	sonar-token

--> after saving

Jenkins

Manage Jenkins / Tools

Gradle installations

+ Add Gradle

SonarScanner for MSBuild installations

+ Add SonarScanner for MSBuild

SonarQube Scanner installations

SonarQube Scanner installations Edited

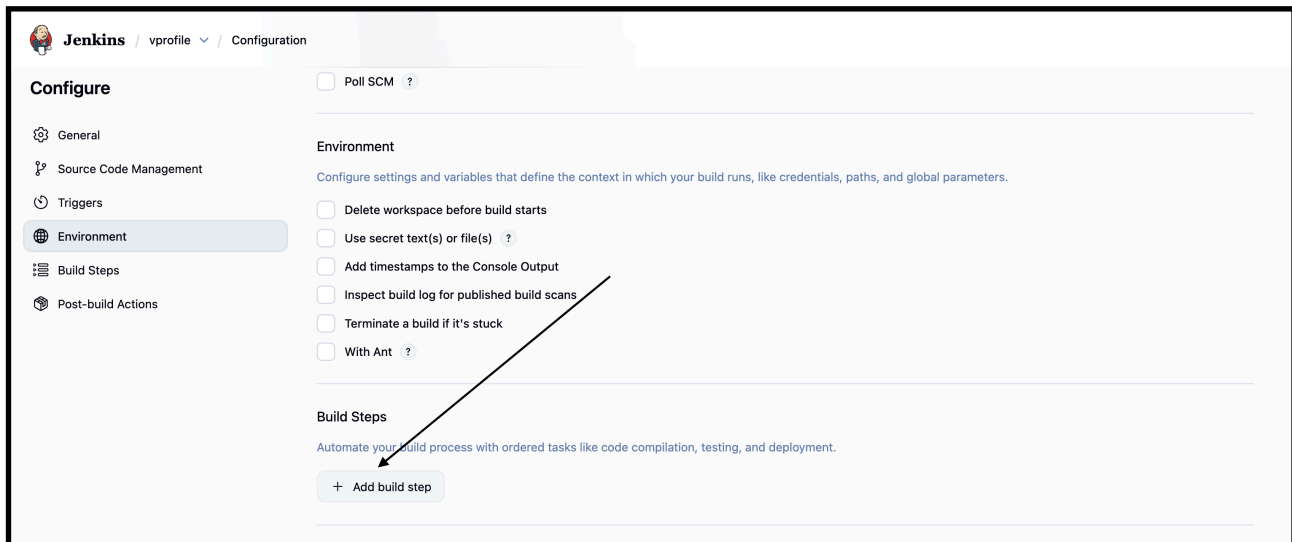
--> go to your vprofile job and

Step-5: Configure Project for Sonar Analysis

Edit your Jenkins job where VProfile Git repo is configured

➔ **Configure Job**

➔ Add a new build step:



sonar.projectKey=Sabear

sonar.projectName=Sabear

sonar.projectVersion=1.0

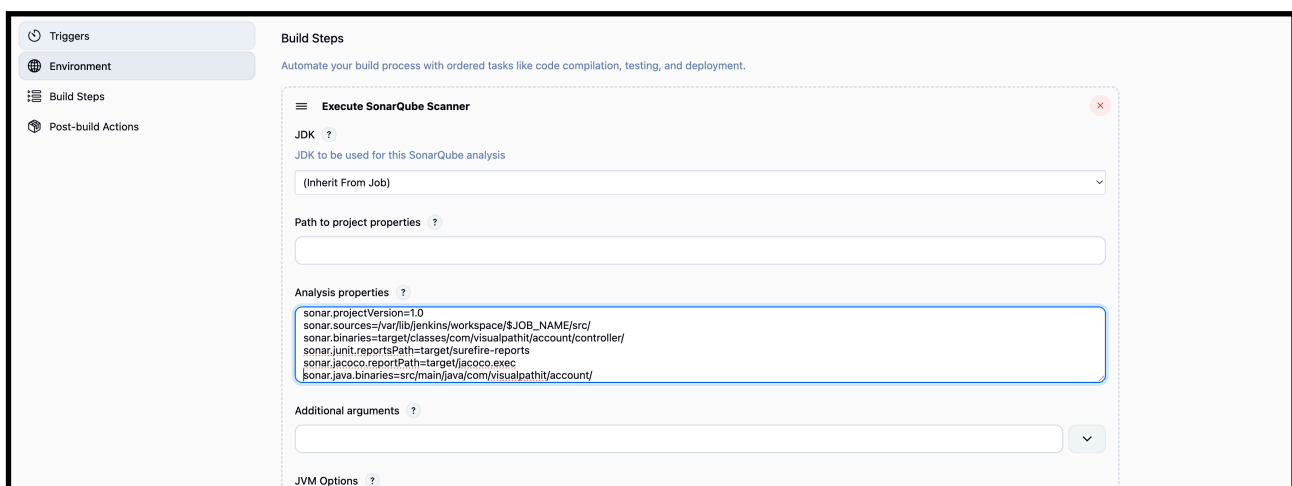
sonar.sources=/var/lib/jenkins/workspace/\$JOB_NAME/src/

sonar.binaries=target/classes/com/visualpathit/account/controller/

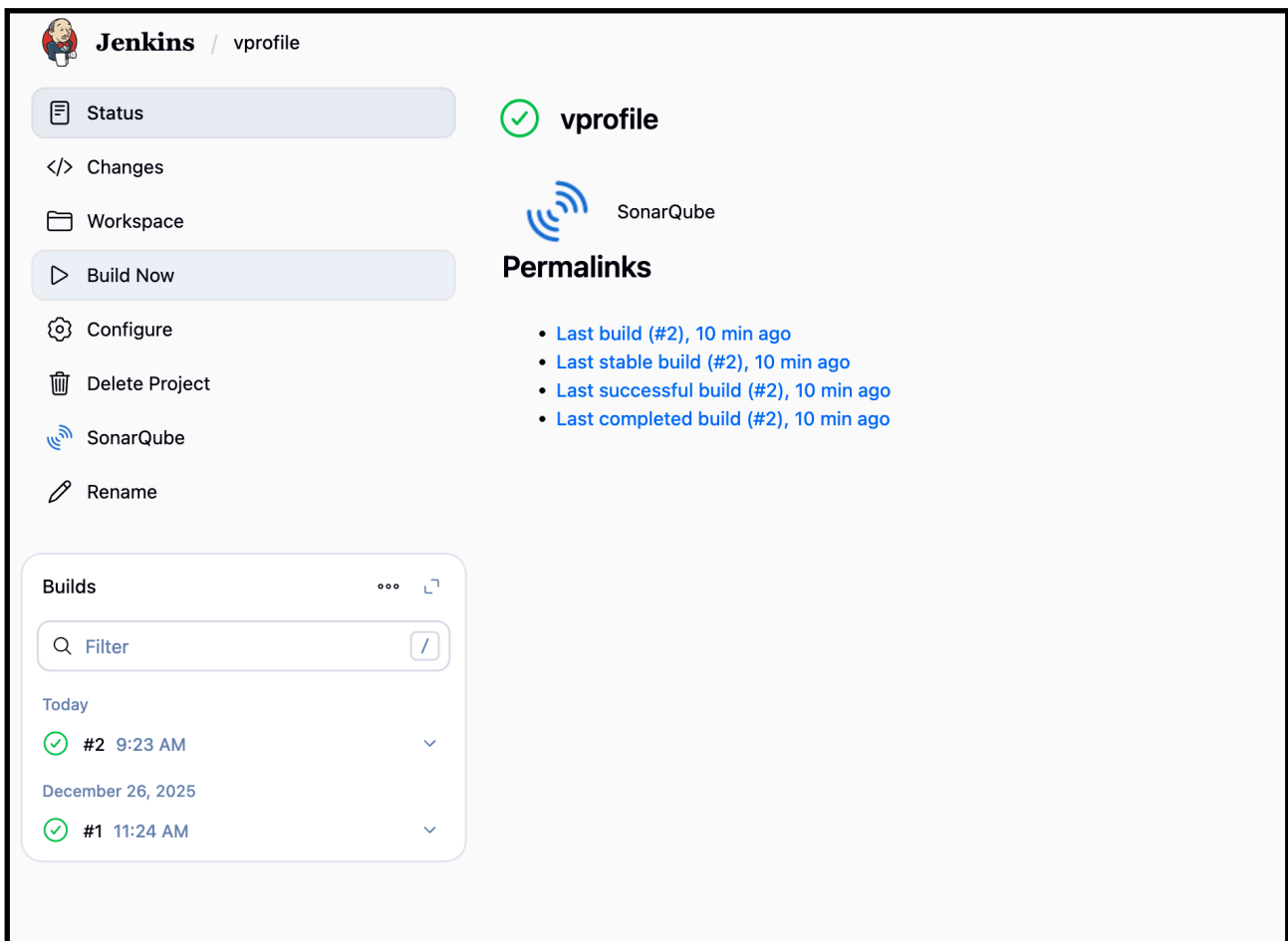
sonar.junit.reportsPath=target/surefire-reports

sonar.jacoco.reportPath=target/jacoco.exec

sonar.java.binaries=src/main/java/com/visualpathit/account/



—> save



The screenshot shows the Jenkins web interface for a job named 'vprofile'. On the left is a sidebar with navigation options: Status (selected), Changes, Workspace, Build Now (highlighted with a play icon), Configure, Delete Project, SonarQube, and Rename. The main content area shows a green checkmark icon next to the job name 'vprofile'. Below this is the SonarQube logo and a section titled 'Permalinks' containing four links: 'Last build (#2), 10 min ago', 'Last stable build (#2), 10 min ago', 'Last successful build (#2), 10 min ago', and 'Last completed build (#2), 10 min ago'. At the bottom left, a 'Builds' panel is visible, featuring a search filter and a list of recent builds. The first build is '#2' at '9:23 AM' with a green status icon, and the second is '#1' at '11:24 AM' also with a green status icon. Both builds have a dropdown arrow to their right.

—->

Step 6 — Build Vprofile Job 🚀

Click **Build Now**

Job should push code to SonarQube → You will see your project in Sonar UI

 Jenkins / vprofile

Status

Changes

Workspace

Build Now

Configure

Delete Project

SonarQube

Rename

 vprofile

 SonarQube

SonarQube Quality Gate

Sabear

Passed

server-side processing:

Success

Permalinks

- Last build (#4), 20 min ago
- Last stable build (#4), 20 min ago
- Last successful build (#4), 20 min ago
- Last completed build (#4), 20 min ago

Builds

Filter

Today

✓

#4

10:08 AM



▼

✓

#2

9:23 AM

▼

December 26, 2025

✓

#1

11:24 AM

▼

 Jenkins / vprofile / #4 / Console Output

Status

Changes

Console Output

Edit Build Information

Delete Build '#4'

Timings

Git Build Data

Previous Build

✓ Console Output

Download

Copy

View

```
Started by user waseem akram
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/vprofile
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/vprofile/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/betawins/VProfile-1.git # timeout=10
Fetching upstream changes from https://github.com/betawins/VProfile-1.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/VProfile-1.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision afc137167b41aaaffaeaf82b81aee721ec883901 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f afc137167b41aaaffaeaf82b81aee721ec883901 # timeout=10
Commit message: "Update README.md"
> git rev-list --no-walk afc137167b41aaaffaeaf82b81aee721ec883901 # timeout=10
[vprofile] $ /var/lib/jenkins/tools/hudson.plugins.sonar.SonarRunnerInstallation/SonarScanner/bin/sonar-scanner -
Dsonar.host.url=http://52.87.178.31:9000 ***** -Dsonar.projectKey=Sabear -Dsonar.projectName=Sabear -Dsonar.jacoco.reportPath=target/jacoco.e
Dsonar.projectVersion=1.0 -Dsonar.sources=/var/lib/jenkins/workspace/vprofile/src/ -Dsonar.java.binaries=src/main/java/com/visualpathit/account/
Dsonar.binaries=target/classes/com/visualpathit/account/controller/ -Dsonar.junit.reportsPath=target/surefire-reports -
Dsonar.projectBaseDir=/var/lib/jenkins/workspace/vprofile
10:08:34.032 INFO Scanner configuration file: /var/lib/jenkins/tools/hudson.plugins.sonar.SonarRunnerInstallation/SonarScanner/conf/sonar-
scanner.properties
10:08:34.035 INFO Project root configuration file: NONE
10:08:34.043 INFO SonarScanner CLI 8.0.1.6346
10:08:34.045 INFO Linux 6.1.158-180.294.amzn2023.x86_64 amd64
10:08:35.066 INFO Communicating with SonarQube Community Build 25.12.0.117093
10:08:35.067 INFO JRE provisioning: os[linux], arch[x86_64]
10:08:35.715 INFO Starting SonarScanner Engine...
10:08:35.715 INFO Java 21.0.9 Eclipse Adoptium (64-bit)
10:08:37.127 INFO Load global settings
10:08:37.249 INFO Load global settings (done) | time=122ms
10:08:37.252 INFO Server id: 1478411E-A71d1b0v-41vdZiUK5fmRK
```

—-> When done → Go to SonarQube dashboard → Check project results

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

SonarQube community

Projects Issues Rules Quality Profiles Quality Gates Administration More

Sabear / main

Overview Issues Security Hotspots Code Measures Activity

Project Settings Project Information

main 7.4k Lines of Code · Version 1.0 Set as homepage

Quality Gate **Passed** Last analysis 1 minute ago

The last analysis has warnings. [See details](#)

New Code Overall Code

Security	Reliability	Maintainability
0 Open issues	32 Open issues	60 Open issues
Accepted issues	Coverage	Duplications
0	0.0%	2.6%
Valid issues that were not fixed	On 164 lines to cover.	On 10k lines.
Security Hotspots		
4		

Get the most out of SonarQube Community Build! Take advantage of the whole ecosystem by using SonarQube for IDE, a free IDE plugin that helps you find and fix issues earlier in your workflow. Connect SonarQube for IDE to SonarQube Community Build to sync rule sets and issue states. [Learn More](#) [Dismiss](#)

Jenkins / vprofile

Status

Changes

Workspace

Build Now

Configure

Delete Project

SonarQube

Rename

Builds

Filter

Today

- #4 10:08 AM
- #2 9:23 AM

December 26, 2025

vprofile

SonarQube

SonarQube Quality Gate

Sabear **Passed**

server-side processing: **Success**

Permalinks

- Last build (#4), 14 min ago
- Last stable build (#4), 14 min ago
- Last successful build (#4), 14 min ago
- Last completed build (#4), 14 min ago

Conclusion:

successfully integrated Jenkins → SonarQube using Docker

Stage-3: Slack Integration to Send Alerts

Objective

Send Jenkins build notifications (success/failure) to a Slack channel.

Prerequisites

Requirement	Details
Slack App installed	With webhook/token access
Slack Workspace	Already created

Configuration Steps

Step1.

—> **Install Slack Notification Plugin in Jenkins**

Go to:

Manage Jenkins → Manage Plugins → Available
Search for:

✓ **Slack Notification Plugin**

Click → Install and Restart Jenkins if required

Jenkins

Manage Jenkins

Plugins

Search

sl

Install

Updates

2

Available plugins

Installed plugins

Advanced settings

Install	Name ↓	Released	Health
☒	Slack Notification795.v4b_9705b_e6d47 slackBuild Notifiers Integrates Jenkins with Slack, allows publishing build statuses, messages and files to Slack channels.	4 mo 26 days ago	100
☐	Job DSL1.93 Build Tools This plugin allows Jobs and Views to be defined via DSLs This plugin is up for adoption! We are looking for new maintainers. Visit our Adopt a Plugin initiative for more	6 mo 26 days ago	80

Jenkins

Manage Jenkins

Plugins

Plugins

Updates

2

Available plugins

Installed plugins

Advanced settings

Download progress

Download progress

Preparation

• Checking internet connectivity

• Checking update center connectivity

• Success

Apache HttpComponents Client 5.x API

✓ Success

Slack Notification

✓ Success

Loading plugin extensions

✓ Success

→ [Go back to the top page](#)

(you can start using the installed plugins right away)

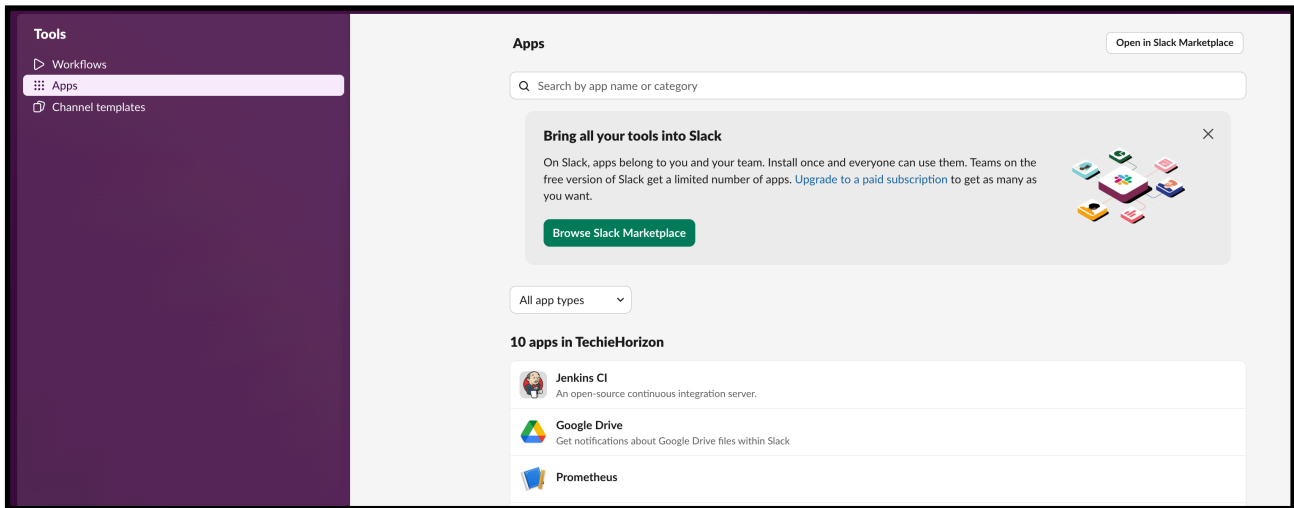
→ ☐ Restart Jenkins when installation is complete and no jobs are running

Step2.

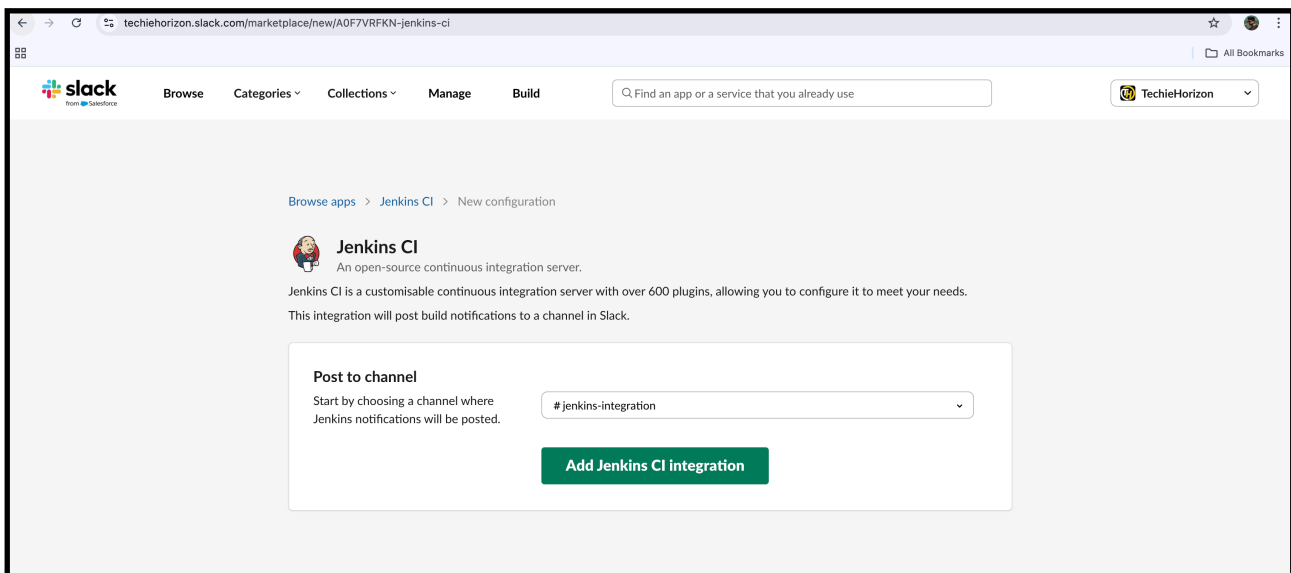
Jenkins + Slack Integration

► Go to Slack:
Go to slack open apps

Select Jenkins cli



—-> add the Jenkins cli



—-> post the channel name #jenkins-intergration

Step 3. Configure Slack in Jenkins

- **Workspace:** your Slack team domain (e.g., myworkspace.slack.com).
- **Integration Token Credential ID:** Add a new credential → Kind = “Secret Text” → Paste URL.
- **Default Channel:** #jenkins-integration
- Save.

The screenshot shows the 'Jenkins Credentials Provider: Jenkins' dialog box with the 'Add Credentials' tab selected. The form contains the following fields:

- Domain:** A dropdown menu with 'Global credentials (unrestricted)' selected.
- Kind:** A dropdown menu with 'Secret text' selected.
- Scope:** A dropdown menu with 'Global (Jenkins, nodes, items, all child items, etc)' selected.
- Secret:** A text input field containing a series of dots, indicating a masked password.
- ID:** A text input field containing the word 'slack'.
- Description:** A text input field containing the word 'slack'.

At the bottom right of the dialog are 'Cancel' and 'Add' buttons. The background shows a blurred view of the Jenkins interface with a sidebar on the left and a main content area on the right.

—-> create a credential for slack

The screenshot shows the 'Integration settings' page for a Slack integration. The page has a light gray background and a white content area. The settings are organized into sections:

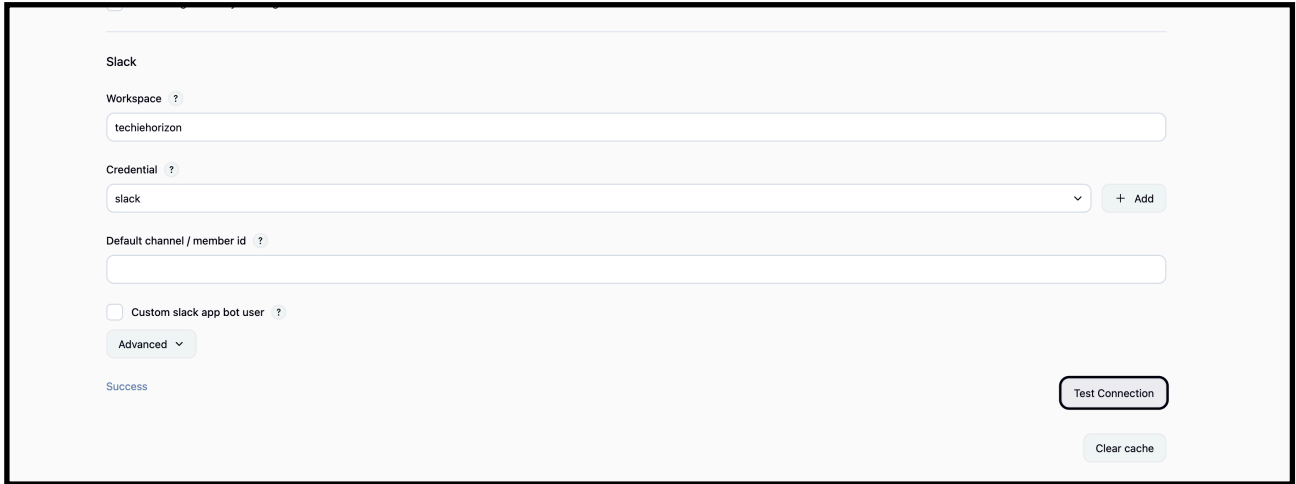
- Post to channel:** A section with the text 'Notifications that come in from Jenkins CI will be posted here.' and a dropdown menu showing '#jenkins-integration'.
- Token:** A section with the text 'This token is used as the key to your Jenkins CI integration.' and a text input field containing 'AttBrNtBEDREXD8U7PtW711q'. A 'Regenerate' link is visible to the right of the field.
- Descriptive label:** A section with the text 'Use this label to provide extra context in your list of integrations (optional).' and a text input field containing 'Optional description of this integration'. A 'Click to go back, hold to see history' link is visible below the field.
- Customise name:** A section with the text 'Choose the username that this integration will post as.' and a text input field containing 'jenkins'.
- Customise icon:** A section with the text 'Change the icon that is used for messages from this integration.' and an 'Upload an image' button.

A green 'Save settings' button is located at the bottom center of the page.

—-> from Jenkins cli copy the token and paste in credentials secret

—> and save

— —> now go to Jenkins configuration and add scroll down to slack conf



Slack

Workspace ?
techiehorizon

Credential ?
slack

Default channel / member id ?

☐ Custom slack app bot user ?

Advanced ▾

Success

Test Connection

Clear cache

—-> here add the workspace name of channel and add the credential
And save and test connection

Step 4 :Add Slack Notifications to Jenkins Job

—-> Add Slack Notifications in Jenkins Job

Open your same Jenkins job → Configure:

Scroll to **Post-Build Actions**

Click:

➡ **Add post-build action → Slack Notifications**

Select notifications:

- ✓ Notify Success
- ✓ Notify Failure
- ✓ Notify Always (optional)

Save job

 Jenkins

vprofile

Configuration

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Analysis properties ?

sonar.projectKey=Sabear

sonar.projectName=Sabear

sonar.projectVersion=1.0

sonar.sources=/var/lib/jenkins/workspace/\$JOB_NAME/src/

sonar.binaries=target/classes/com/visualpathit/account/controller/

sonar.junit.reportsPath=target/surefire-reports

sonar.jacoco.reportPath=target/jacoco.exec

Filter

Aggregate downstream test results

Archive the artifacts

Build other projects

Publish JUnit test result report

Record fingerprints of files to track usage

Git Publisher

E-mail Notification

Editable Email Notification

Set GitHub commit status (universal)

Set build status on GitHub commit [deprecated]

Slack Notifications

Delete workspace when build is done

+ Add post-build action

Save

Apply

 Jenkins

vprofile

Configuration

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

JVM Options ?

+ Add build step

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Slack Notifications

☒ Notify Build Start

☒ Notify Success

☐ Notify Aborted

☐ Notify Not Built

☐ Notify Unstable

☐ Notify Regression

☐ Notify Every Failure

☐ Notify Back To Normal

Advanced

Save

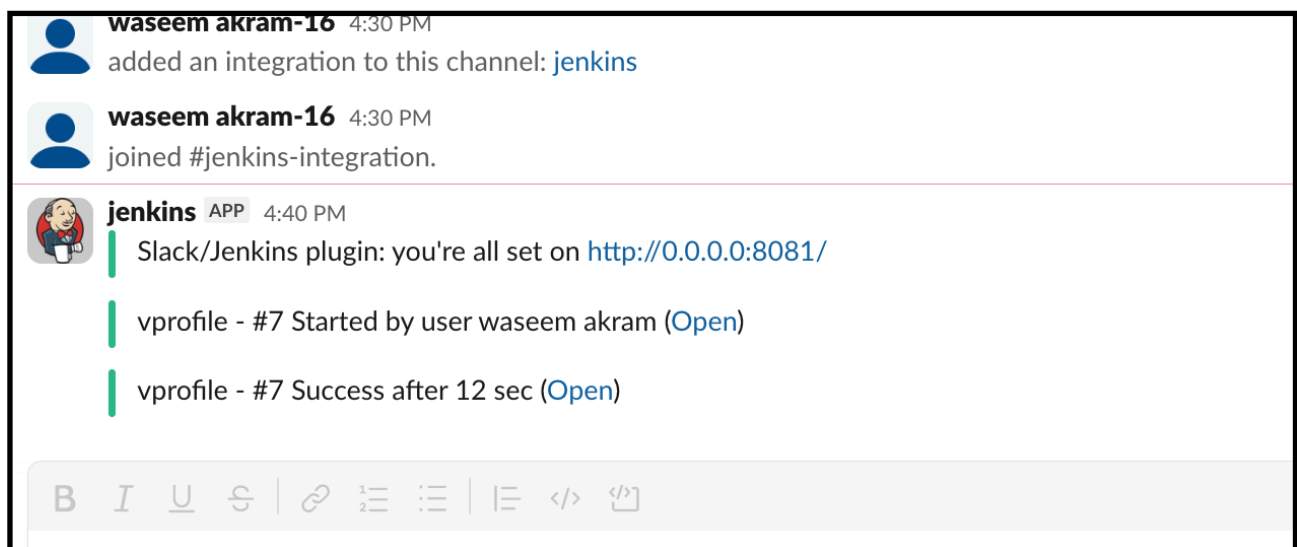
Apply

Testing Stage-3

- 1 Trigger the Jenkins job manually
- 2 Check your Slack channel
- 3 You should see notifications:

Example:

- > Build SUCCESS
- > Build FAILED



Conclusion :

Stage-3 Output

- ✓ Slack alerts received for every Jenkins build
- ✓ Shows build status and job name
- ✓ Confirm integration success

Task 2 : similarly from task 1

1) Create one jenkins job using the below code and create three stages.

stage1: Git clone to download the source code.

stage2: Sonarqube Integration to check the quality of code

stage3: Slack Integration to send the alerts to slack.

URL: <https://github.com/betawins/hiring-app.git>

stage1: Git clone to download the source code.

The screenshot shows the Jenkins Configuration page for a job named 'hiring-app'. The left sidebar contains a 'Configure' section with a list of tabs: General, Source Code Management, Triggers, Environment, Build Steps, and Post-build Actions. The 'Source Code Management' tab is selected. The main content area is titled 'Source Code Management' and includes a description: 'Connect and manage your code repository to automatically pull the latest code for your builds.' There are two radio buttons: 'None' and 'Git'. The 'Git' option is selected. Below this, there is a 'Repositories' section with a table containing one repository. The 'Repository URL' is 'https://github.com/betawins/hiring-app.git' and the 'Credentials' are set to '- none -'. There is an 'Add Repository' button. Below the repositories, there is a 'Branches to build' section with a 'Branch Specifier (blank for 'any')' field set to '*/master'. At the bottom, there are 'Save' and 'Apply' buttons.

Repository URL	Credentials
https://github.com/betawins/hiring-app.git	- none -

Follow the similar steps and configurations as above task 1

```
[root@ip-172-31-65-24 jenkins]# cd workspace
[root@ip-172-31-65-24 workspace]# ls
first-job hiring-app job2 job3 private private@tmp vprofile
[root@ip-172-31-65-24 workspace]# cd hiring-app
[root@ip-172-31-65-24 hiring-app]# ll
total 0
[root@ip-172-31-65-24 hiring-app]# ll
total 24
-rw-r--r--. 1 jenkins jenkins 476 Dec 29 11:59 Dockerfile
-rw-r--r--. 1 jenkins jenkins 1757 Dec 29 11:59 Jenkinsfile
-rw-r--r--. 1 jenkins jenkins 88 Dec 29 11:59 README.md
-rw-r--r--. 1 jenkins jenkins 408 Dec 29 11:59 'Untitled Diagram.drawio'
-rw-r--r--. 1 jenkins jenkins 1801 Dec 29 11:59 jenkinsfile-cicd
-rw-r--r--. 1 jenkins jenkins 890 Dec 29 11:59 pom.xml
drwxr-xr-x. 3 jenkins jenkins 18 Dec 29 11:59 src
[root@ip-172-31-65-24 hiring-app]#
```

stage2: Sonarqube Integration to check the quality of code

Installed new server for sonarqube using postgresql

Step 1 :

- > launch a instance and
- > install java

```

,      #_
~\    #####      Amazon Linux 2023
~~~\  #####\
~~~~\#####\
~~~~\###|
~~~~\#/\_==
~~~~V~' '->
~~~~
~~~~_._\_/
~~~~\_/ \_/
~~~~/m/'

```

Dependencies resolved.

Package	Architecture
Installing:	
java-17-amazon-corretto	x86_64
Installing dependencies:	
alsa-lib	x86_64
cairo	x86_64
dejavu-sans-fonts	noarch
dejavu-sans-mono-fonts	noarch

```
--> sudo dnf install postgresql15 postgresql15-server -y
```

Dependencies resolved.

Package	Architecture
Installing:	
postgresql15	x86_64
postgresql15-server	x86_64
Installing dependencies:	
libc	x86_64
postgresql15-private-libs	x86_64

Install 4 Packages

```
(4/4): postgresql15-server-15.15-1.amzn2023.0.1.x86_64.rpm
```

Transaction check succeeded

Step 3 : Initialize PostgreSQL Database

```
sudo /usr/bin/postgresql-setup --initdb
```

```
[root@ip-172-31-95-0 ~]# sudo /usr/bin/postgresql-setup --initdb
* Initializing database in '/var/lib/pgsql/data'
* Initialized, logs are in /var/lib/pgsql/initdb_postgresql.log
[root@ip-172-31-95-0 ~]#
```

Step 4 : Enable & Start PostgreSQL Service

```
sudo systemctl enable postgresql
sudo systemctl start postgresql
sudo systemctl status postgresql --no-pager
```

```
sudo systemctl enable postgresql
[root@ip-172-31-95-0 ~]# sudo systemctl status postgresql --no-pager
● postgresql.service - PostgreSQL database server
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-12-29 12:31:39 UTC; 16s ago
     Main PID: 27895 (postgres)
        Tasks: 7 (limit: 9307)
       Memory: 17.2M
          CPU: 30ms
   CGroup: /system.slice/postgresql.service
           └─27895 /usr/bin/postgres -D /var/lib/pgsql/data
           └─27896 "postgres: logger "
           └─27897 "postgres: checkpointer "
           └─27898 "postgres: background writer "
           └─27900 "postgres: walwriter "
           └─27901 "postgres: autovacuum launcher "
           └─27902 "postgres: logical replication launcher "
```

```
Dec 29 12:31:39 ip-172-31-95-0.ec2.internal systemd[1]: Starting postgresql.service - PostgreSQL database server...
Dec 29 12:31:39 ip-172-31-95-0.ec2.internal postgres[27895]: 2025-12-29 12:31:39.839 UTC [27895] LOG:  redirecting log output to logging collector process
Dec 29 12:31:39 ip-172-31-95-0.ec2.internal postgres[27895]: 2025-12-29 12:31:39.839 UTC [27895] HINT:  Future log output will appear in directory "log".
Dec 29 12:31:39 ip-172-31-95-0.ec2.internal systemd[1]: Started postgresql.service - PostgreSQL database server.
[root@ip-172-31-95-0 ~]#
```

Step 5 : Login to PostgreSQL

```
sudo -u postgres psql
```

—>Create Database for SonarQube

Inside PostgreSQL prompt:

```
CREATE DATABASE sonarqube;
```

—->Create SonarQube User

```
CREATE USER sonar WITH ENCRYPTED PASSWORD 'sonar123';
```

—> Grant Privileges

```
GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonar;
```

—>Exit from PostgreSQL

```
\q
```

—>important information always remember the user and password
Given in this data base

```
[root@ip-172-31-95-0 ~]# sudo -u postgres psql
could not change directory to "/root": Permission denied
psql (15.15)
Type "help" for help.

postgres=# CREATE DATABASE sonarqube;
CREATE DATABASE
postgres=# CREATE USER sonar WITH ENCRYPTED PASSWORD 'sonar@123';
CREATE ROLE
postgres=# GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonar;
GRANT
postgres=# ALTER USER sonar WITH SUPERUSER;
ALTER ROLE
postgres=# \q
[root@ip-172-31-95-0 ~]#
```

Step 6 — Allow Authentication in PostgreSQL

Edit config:

```
sudo vi /var/lib/pgsql/data/pg_hba.conf
```

Find & change:

```
host      all      all      127.0.0.1/32      md5
```

```
# "local" is for Unix domain socket connections only
local all all peer
# IPv4 local connections:
host all all 127.0.0.1/32 md5
# IPv6 local connections:
host all all ::1/128 ident
# Allow replication connections from localhost, by a user with the
# replication privilege.
local replication all peer
host replication all 127.0.0.1/32 ident
```

Step 7 — Allow External Connections (optional)

```
sudo vi /var/lib/pgsql/data/postgresql.conf
```

```
#-----
# CONNECTIONS AND AUTHENTICATION
#-----

# - Connection Settings -

listen_addresses = '*' # what IP address(es) to listen on;
                        # comma-separated list of addresses;
                        # defaults to 'localhost'; use '*' for all
                        # (change requires restart)
#port = 5432           # (change requires restart)
max_connections = 100  # (change requires restart)
```

```
opensdk on bit server in corrects 172.31.10.1 (data 172.31.10.2), mixed mode
[root@ip-172-31-95-0 ~]# sudo vi /var/lib/pgsql/data/pg_hba.conf
[root@ip-172-31-95-0 ~]# sudo vi /var/lib/pgsql/data/postgresql.conf
[root@ip-172-31-95-0 ~]#
```


—> restart the postgresql
Sudo systemctl restart postgresql

```
[root@ip-172-31-95-0 ~]# sudo systemctl restart postgresql
[root@ip-172-31-95-0 ~]#
```

Step 8 — Install SonarQube

```
cd /opt
sudo wget https://binaries.sonarsource.com/Distribution/
sonarqube/sonarqube-9.9.1.69595.zip
sudo dnf install unzip -y
sudo unzip sonarqube-9.9.1.69595.zip
sudo mv sonarqube-9.9.1.69595 sonarqube
```

—> cd /opt
sudo wget https://binaries.sonarsource.com/Distribution/
sonarqube/sonarqube-9.9.1.69595.zip

```
[root@ip-172-31-95-0 ~]# cd /opt
[root@ip-172-31-95-0 opt]# sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-9.9.1.69595.zip
--2025-12-29 13:11:41-- https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-9.9.1.69595.zip
Resolving binaries.sonarsource.com (binaries.sonarsource.com)... 99.84.188.21, 99.84.188.106, 99.84.188.45, ...
Connecting to binaries.sonarsource.com (binaries.sonarsource.com)|99.84.188.21|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 293875683 (280M) [binary/octet-stream]
Saving to: 'sonarqube-9.9.1.69595.zip'

sonarqube-9.9.1.69595.zip                               100%[=====]
2025-12-29 13:11:42 (360 MB/s) - 'sonarqube-9.9.1.69595.zip' saved [293875683/293875683]

[root@ip-172-31-95-0 opt]#
```

—>sudo dnf install unzip -y
—>sudo unzip sonarqube-9.9.1.69595.zip

```
[root@ip-172-31-95-0 opt]# sudo dnf install unzip -y
Last metadata expiration check: 0:56:36 ago on Mon Dec 29 12:16:33 2025.
Package unzip-6.0-57.amzn2023.0.2.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-95-0 opt]# ls
aws  sonarqube-9.9.1.69595.zip
[root@ip-172-31-95-0 opt]# sudo unzip sonarqube-9.9.1.69595.zip
Archive:  sonarqube-9.9.1.69595.zip
  creating: sonarqube-9.9.1.69595/
  inflating: sonarqube-9.9.1.69595/dependency-license.json
  inflating: sonarqube-9.9.1.69595/COPYING
  creating: sonarqube-9.9.1.69595/bin/
```

-->sudo mv sonarqube-9.9.1.69595 sonarqube

Create sonar user:

—>Create SonarQube User & Group

```
sudo groupadd sonar
sudo useradd -d /opt/sonarqube -g sonar sonar
```

— > Set Permissions

```
sudo chown -R sonar:sonar /opt/sonarqube
```

```
[root@ip-172-31-95-0 opt]# sudo mv sonarqube-9.9.1.69595 sonarqube
[root@ip-172-31-95-0 opt]# sudo groupadd sonar
sudo useradd -d /opt/sonarqube -g sonar sonar
sudo chown -R sonar:sonar /opt/sonarqube
useradd: warning: the home directory /opt/sonarqube already exists.
useradd: Not copying any file from skel directory into it.
[root@ip-172-31-95-0 opt]# ll
total 286988
drwxr-xr-x.  4 root  root      33 Dec  3 16:42 aws
drwxr-xr-x. 11 sonar sonar    172 Apr 27  2023 sonarqube
-rw-r--r--.  1 root  root 293875683 Apr 28  2023 sonarqube-9.9.1.69595.zip
[root@ip-172-31-95-0 opt]#
```

Step 9 — Configure SonarQube Database Connection

```
sudo vi /opt/sonarqube/conf/sonar.properties
```

Find and update these lines:

```
sonar.jdbc.username=sonar
sonar.jdbc.password=sonar123
sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube
```

Also set **web port**:

```
sonar.web.host=0.0.0.0
sonar.web.port=9000
```

```
#-----
# DATABASE
#
# IMPORTANT:
# - The embedded H2 database is used by default. It is recommended for tests but not for
#   production use. Supported databases are Oracle, PostgreSQL and Microsoft SQLServer.
# - Changes to database connection URL (sonar.jdbc.url) can affect SonarSource licensed products.
#
# User credentials.
# Permissions to create tables, indices and triggers must be granted to JDBC user.
# The schema must be created first.
sonar.jdbc.username=sonar
sonar.jdbc.password=Sonar123
#----- Embedded Database (default)
# H2 embedded database server listening port, defaults to 9092
#sonar.embeddedDatabase.port=9092
#
#----- Oracle 19c/21c
```

```

# The schema must be created first.
sonar.jdbc.username=sonar
sonar.jdbc.password=Sonar123

#----- Embedded Database (default)
# H2 embedded database server listening port, defaults to 9092
#sonar.embeddedDatabase.port=9092

#----- Oracle 19c/21c
# The Oracle JDBC driver must be copied into the directory extensions/jdbc-driver/oracle/.
# Only the thin client is supported, and we recommend using the latest Oracle JDBC driver. See
# https://jira.sonarsource.com/browse/SONAR-9758 for more details.
# If you need to set the schema, please refer to http://jira.sonarsource.com/browse/SONAR-5000
#sonar.jdbc.url=jdbc:oracle:thin:@localhost:1521/XE

#----- PostgreSQL 11 or greater
# By default the schema named "public" is used. It can be overridden with the parameter "currentSchema".
sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube

```

```

#-----
# WEB SERVER
# Web server is executed in a dedicated Java process. By default heap size is 512MB.
# Use the following property to customize JVM options.
#   Recommendations:
#
#   The HotSpot Server VM is recommended. The property -server should be added if server mode
#   is not enabled by default on your environment:
#   http://docs.oracle.com/javase/8/docs/technotes/guides/vm/server-class.html
#
#   Startup can be long if entropy source is short of entropy. Adding
#   -Djava.security.egd=file:/dev/./urandom is an option to resolve the problem.
#   See https://wiki.apache.org/tomcat/HowTo/FasterStartUp#Entropy_Source
#
#sonar.web.javaOpts=-Xmx512m -Xms128m -XX:+HeapDumpOnOutOfMemoryError

# Same as previous property, but allows to not repeat all other settings like -Xmx
#sonar.web.javaAdditionalOpts=

# Binding IP address. For servers with more than one IP address, this property specifies which
# address will be used for listening on the specified ports.
# By default, ports will be used on all IP addresses associated with the server.
sonar.web.host=0.0.0.0

# Web context. When set, it must start with forward slash (for example /sonarqube).
# Changing this value and restarting the server can have unexpected consequences for logged in users,
# as they may end up with multiple conflicting browser cookies. If your users experience odd
# behaviors after you changed this value and restarted the server, ask them to clear their browser's
# cookies and log in again.
# The default value is root context (empty value).
#sonar.web.context=

# TCP port for incoming HTTP connections. Default value is 9000.
sonar.web.port=9000

# The maximum number of connections that the server will accept and process at any given time.
# When this number has been reached, the server will not accept any more connections until

```

Step 10 — Start SonarQube Manually For First Test

```
sudo -u sonar /opt/sonarqube/bin/linux-x86-64/sonar.sh start
```

```
sudo -u sonar /opt/sonarqube/bin/linux-x86-64/sonar.sh status
```

```
[root@ip-172-31-95-0 opt]# sudo -u sonar /opt/sonarqube/bin/linux-x86-64/sonar.sh start
/usr/bin/java
Starting SonarQube...
Started SonarQube.
[root@ip-172-31-95-0 opt]# sudo -u sonar /opt/sonarqube/bin/linux-x86-64/sonar.sh status
/usr/bin/java
SonarQube is running (30014).
[root@ip-172-31-95-0 opt]#
```

—-> check the port running or not

—-> Sudo ss -tulnp | grep 9000

```
sudo -u sonar /opt/sonarqube/bin/linux-x86-64/sonar.sh start
/usr/bin/java
Gracefully stopping SonarQube...
Removed stale pid file: ./SonarQube.pid
SonarQube was not running.
/usr/bin/java
Starting SonarQube...
Started SonarQube.
[root@ip-172-31-95-0 opt]# sudo ss -tulnp | grep 9000
tcp    LISTEN 0      25          *:9000      *:*    users:(("java",pid=35193,fd=12))
```

Step 11 — Access SonarQube

Open browser:

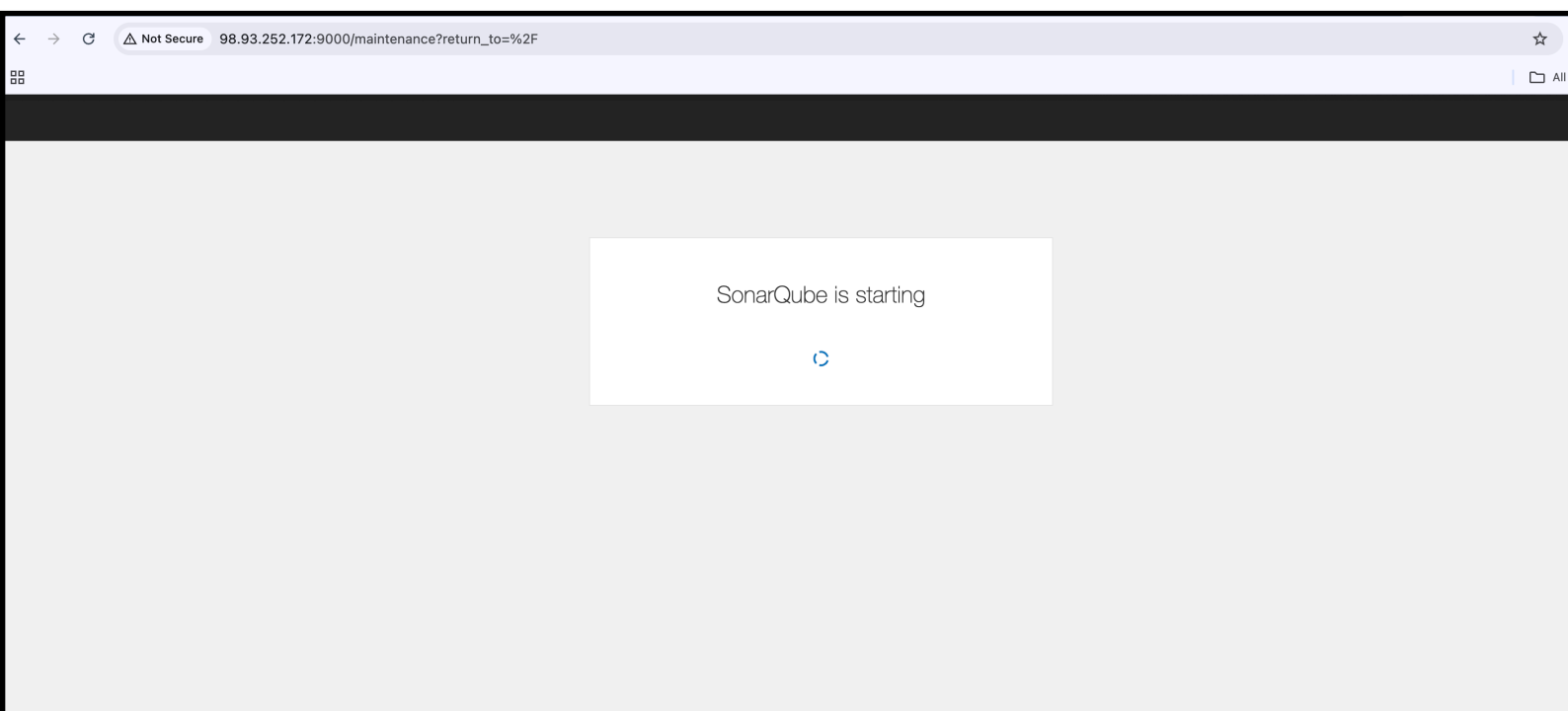
`http://<EC2-Public-IP>:9000`

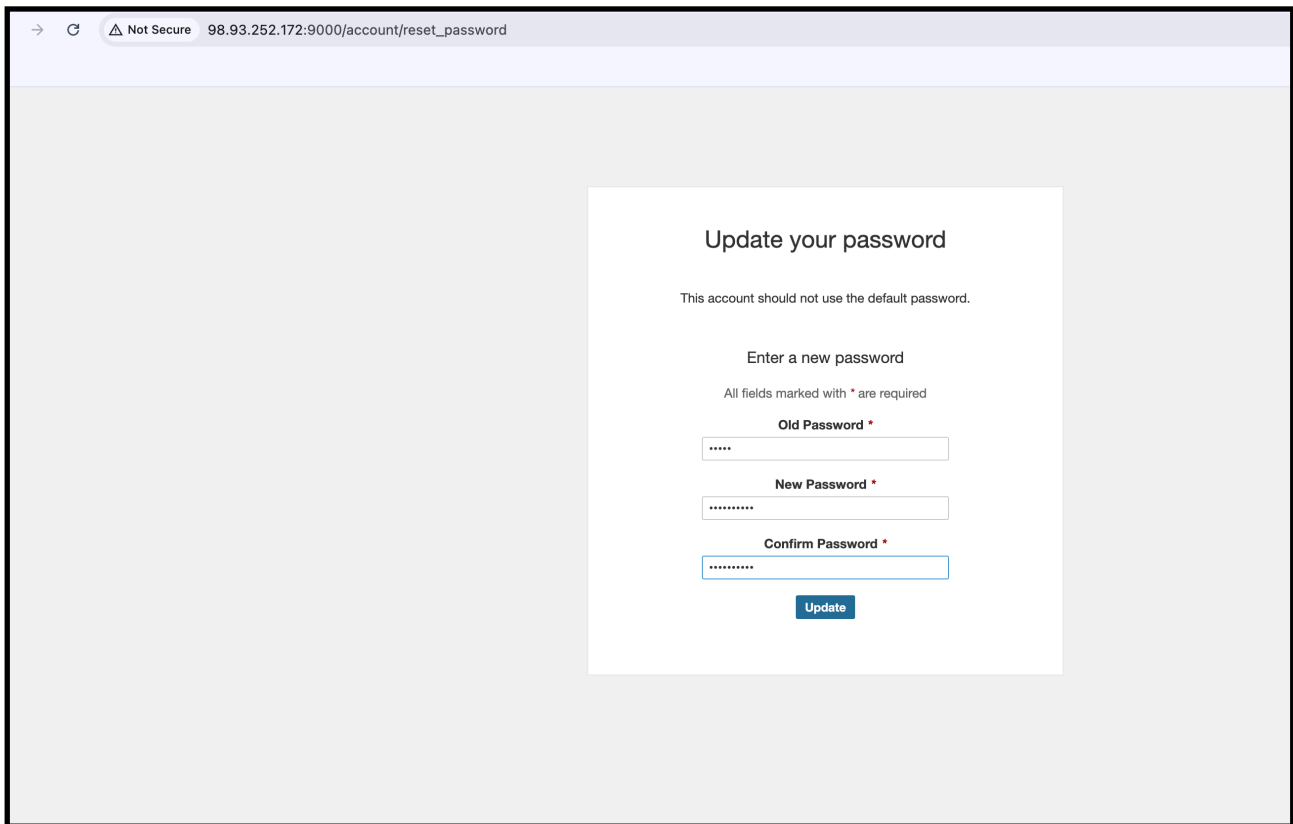
Default login:

Username: admin

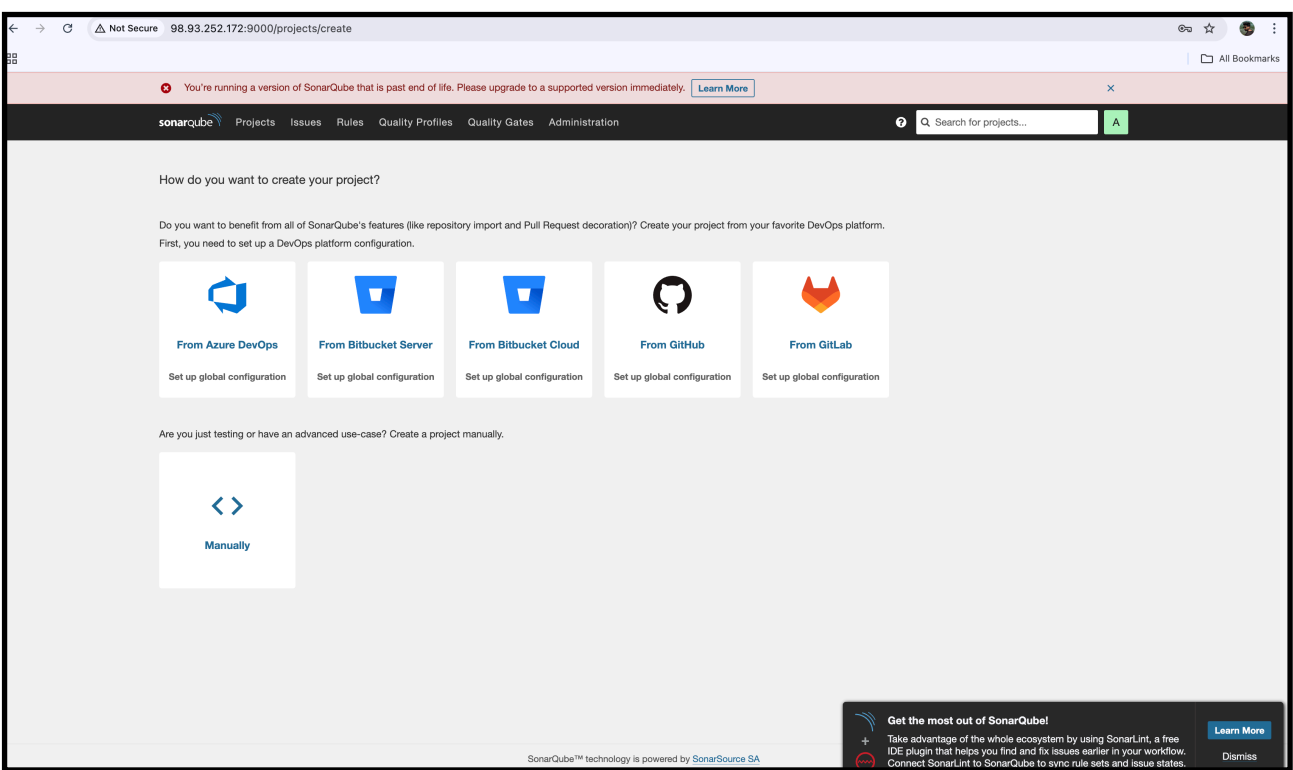
Password: admin

Change password when prompted.





— —> now we have to create a token in security



Generate Tokens

Name: Type: Expires in:

Message: New token "jenkins-sonar" has been created. Make sure you copy it now, you won't be able to see it again!

`sqa_4c3836e11f4e6ed639fd67e9d1445971af50c528`

Name	Type	Project	Last use	Created	Expiration	
jenkins-sonar	Global		Never	December 29, 2025	January 28, 2026	Revoke

- > now go to your Jenkins master
- > create new credentials
- > select the secret text
 - > in side secret paste the token from sonarqube
- > save

Jenkins Credentials Provider: Jenkins

Add Credentials

Domain:

Kind:

Scope:

Secret:

ID:

Description:

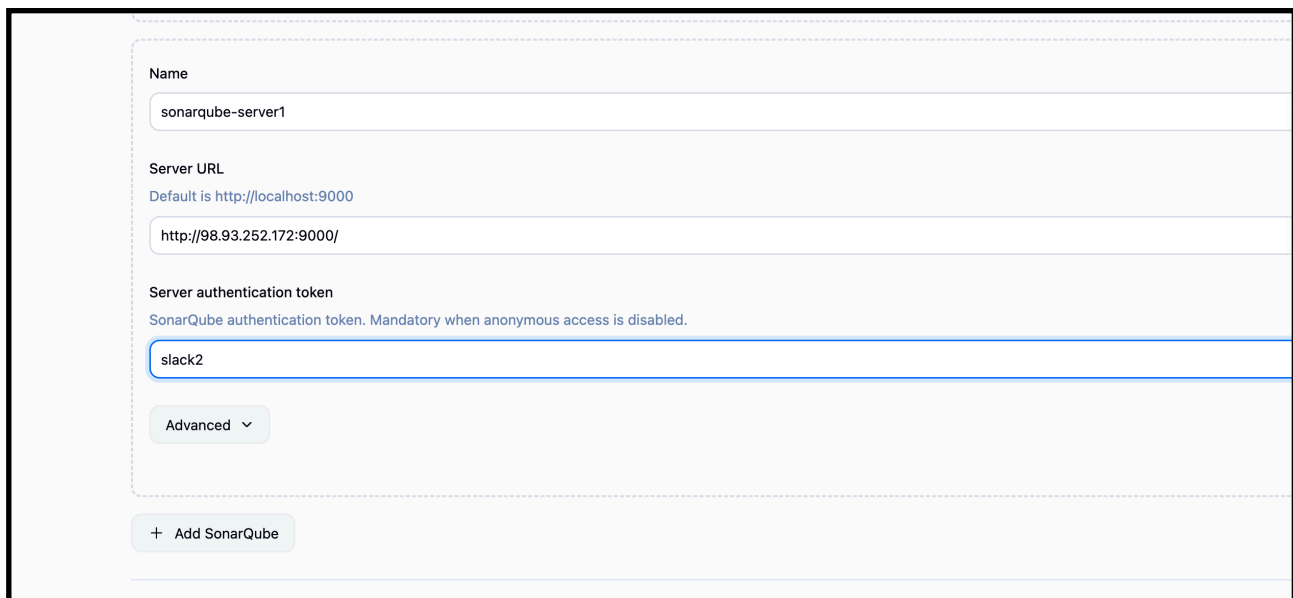
- > after creating the credential
- > go to Jenkins —> manage Jenkins —> plugins —> available plugins
- > search sonarqube scanner
- > and install it

Step 12 : Configure SonarQube Server in Jenkins

Manage Jenkins

- ➔ **System**
- ➔ Scroll to **SonarQube servers**
- ➔ **Add SonarQube**

Fill: the credentials which we created



The screenshot shows the 'Add SonarQube' configuration form in Jenkins. The form is titled 'Name' and contains the following fields:

- Name:** sonarqube-server1
- Server URL:** http://98.93.252.172:9000/ (Default is http://localhost:9000)
- Server authentication token:** slack2 (SonarQube authentication token. Mandatory when anonymous access is disabled.)
- Advanced:** A dropdown menu with a downward arrow.

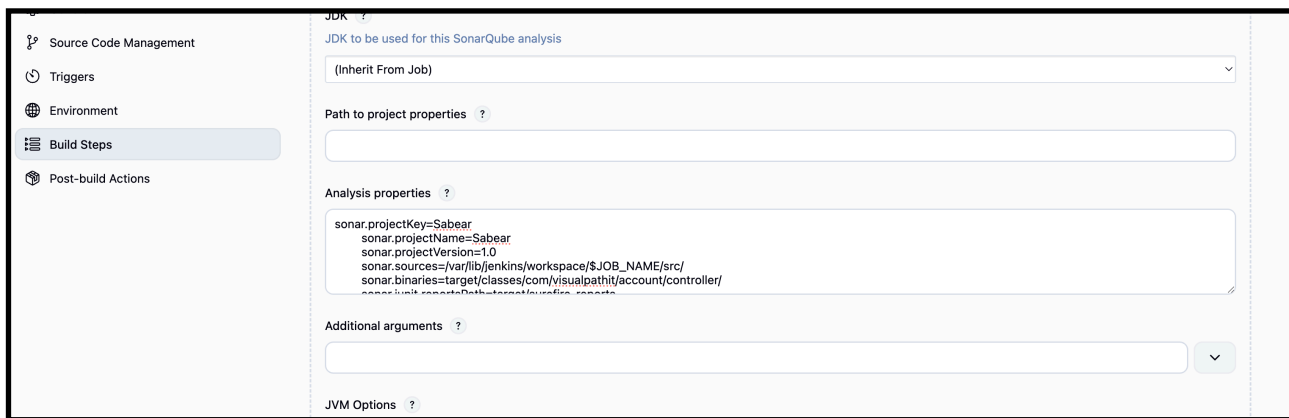
At the bottom of the form, there is a button labeled '+ Add SonarQube'.

Step 13: Configure Project for Sonar Analysis

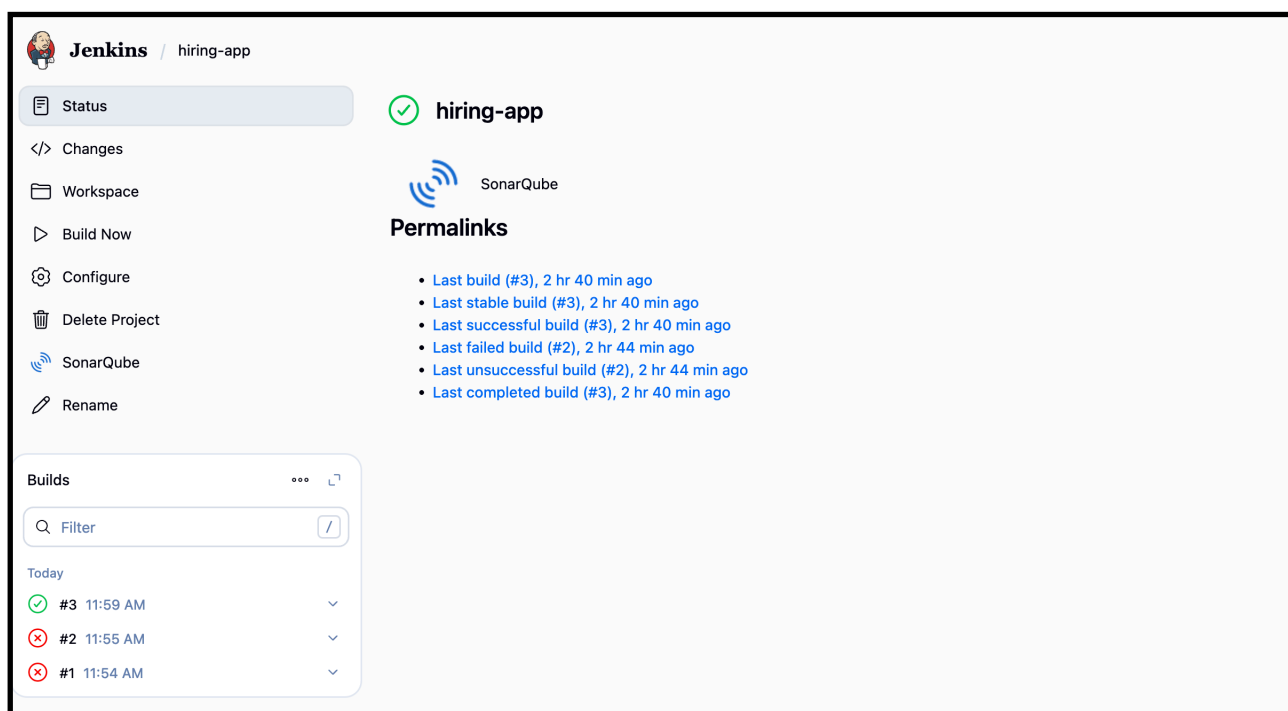
Edit your Jenkins job where hiring-app Git repo is configured

- ➔ **Configure Job**
- ➔ Add a new build step: —>

```
sonar.projectKey=Sabear
sonar.projectName=Sabear
sonar.projectVersion=1.0
sonar.sources=/var/lib/jenkins/workspace/$JOB_NAME/src/
sonar.binaries=target/classes/com/visualpathit/account/controller/
sonar.junit.reportsPath=target/surefire-reports
sonar.jacoco.reportPath=target/jacoco.exec
sonar.java.binaries=src/main/java/com/visualpathit/account/
```



—> save



hiring-app

SonarQube

Permalinks

- [Last build \(#3\), 2 hr 40 min ago](#)
- [Last stable build \(#3\), 2 hr 40 min ago](#)
- [Last successful build \(#3\), 2 hr 40 min ago](#)
- [Last failed build \(#2\), 2 hr 44 min ago](#)
- [Last unsuccessful build \(#2\), 2 hr 44 min ago](#)
- [Last completed build \(#3\), 2 hr 40 min ago](#)

Builds

Build Number	Status	Time
#3	Success	11:59 AM
#2	Failure	11:55 AM
#1	Failure	11:54 AM

Step 14 — Build hiring-app Job

Click **Build Now**

Job should push code to SonarQube → You will see your project in Sonar UI

Delete Project
 SonarQube
 Rename

- Last stable build (#3), 2 hr 41 min ago
- Last successful build (#3), 2 hr 41 min ago
- Last failed build (#2), 2 hr 46 min ago
- Last unsuccessful build (#2), 2 hr 46 min ago
- Last completed build (#3), 2 hr 41 min ago

Builds

Today

✓ #4 2:41 PM

Jenkins

hiring-app

#4

Console Output

Status

Changes

Console Output

Edit Build Information

Delete build '#4'

Timings

Git Build Data

Previous Build

Console Output

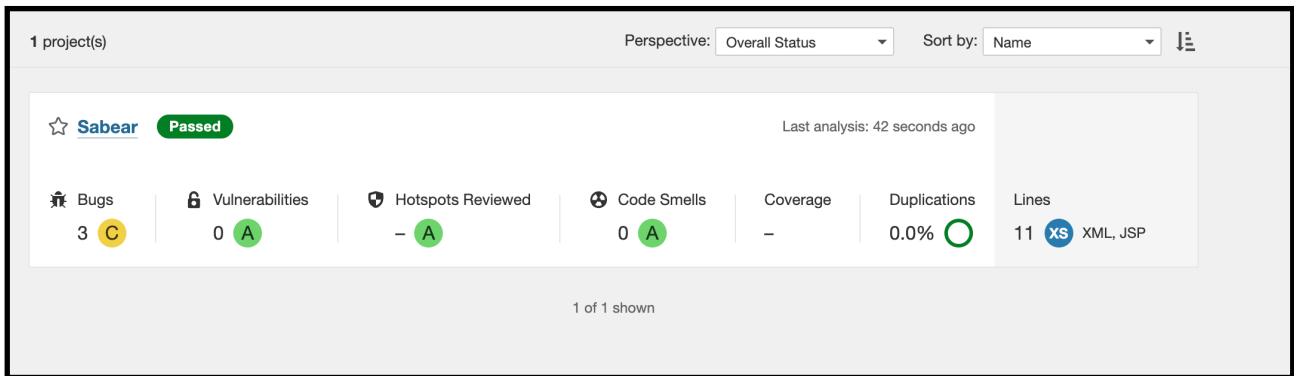
Download Copy View as plain text

```

Started by user waseem akram
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/hiring-app
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/hiring-app/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/betawins/hiring-app.git # timeout=10
Fetching upstream changes from https://github.com/betawins/hiring-app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/hiring-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9405b107fdffdf68e69fe953eaf9669784f9d6a (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9405b107fdffdf68e69fe953eaf9669784f9d6a # timeout=10
Commit message: "Update README.md"
> git rev-list --no-walk 9405b107fdffdf68e69fe953eaf9669784f9d6a # timeout=10
[hiring-app] $ /var/lib/jenkins/tools/hudson.plugins.sonar.SonarRunnerInstallation/SonarScanner/bin/sonar-scanner -
Dsonar.host.url=http://98.93.252.172:9000/ ***** -Dsonar.projectKey=Sabear -Dsonar.projectName=Sabear -Dsonar.jacoco.reportPath=target/jacoco.exec
-Dsonar.projectVersion=1.0 -Dsonar.sources=/var/lib/jenkins/workspace/hiring-app/src/ -Dsonar.java.binaries=src/main/java/com/visualpathit/account/ -
-Dsonar.binaries=target/classes/com/visualpathit/account/controller/ -Dsonar.junit.reportsPath=target/surefire-reports -
-Dsonar.projectBaseDir=/var/lib/jenkins/workspace/hiring-app
14:41:27.830 INFO Scanner configuration file: /var/lib/jenkins/tools/hudson.plugins.sonar.SonarRunnerInstallation/SonarScanner/conf/sonar-
scanner.properties
14:41:27.833 INFO Project root configuration file: NONE
14:41:27.845 INFO SonarScanner CLI 8.0.1.6346
14:41:27.847 INFO Linux 6.1.158-100.294.amzn2023.x86_64 amd64
14:41:28.984 INFO Communicating with SonarQube Server 9.9.1.69595
14:41:29.751 INFO Load global settings
14:41:29.804 INFO Load global settings (done) | time=53ms
14:41:29.806 INFO Server id: 92088F0A-AZtqf_Xvo47IHJdSd50Fg
14:41:29.809 INFO User cache: /var/lib/jenkins/.sonar/cache
14:41:29.811 INFO Load/Download analysis

```

—> When done → Go to SonarQube dashboard → Check project results



Conclusion:

successfully integrated Jenkins → SonarQube

Stage-3: Slack Integration to Send Alerts

Objective

Send Jenkins build notifications (success/failure) to a Slack channel.

—-> follow same steps from task1 stage 3

 **Jenkins** / hiring-app ▾ / Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions**

+ Add build step

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or

Slack Notifications

☒ Notify Build Start

☒ Notify Success

☐ Notify Aborted

☐ Notify Not Built

☐ Notify Unstable

☐ Notify Regression

☐ Notify Every Failure

☐ Notify Back To Normal

Advanced ▾

+ Add post-build action

 **jenkins** APP 8:20 PM

hiring-app - #5 Started by user waseem akram (Open)

hiring-app - #5 Success after 6 sec (Open)

B

T

U

S

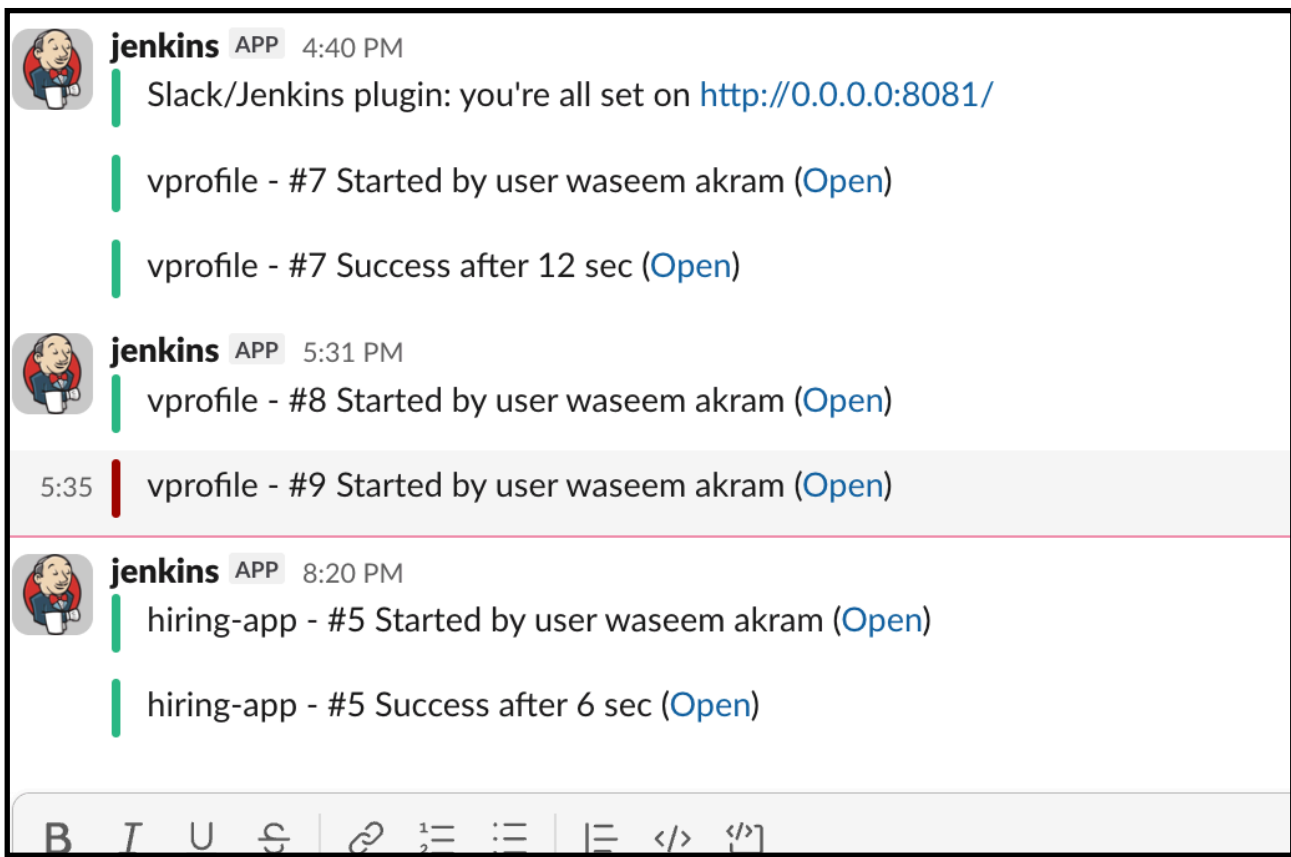
↺

1=

:=

|=

</>



The screenshot shows a Slack interface with a conversation from the 'jenkins' app. The first message, at 4:40 PM, says 'Slack/Jenkins plugin: you're all set on <http://0.0.0.0:8081/>'. It is followed by two green status messages: 'vprofile - #7 Started by user waseem akram (Open)' and 'vprofile - #7 Success after 12 sec (Open)'. The second message, at 5:31 PM, says 'vprofile - #8 Started by user waseem akram (Open)'. Below it, a greyed-out message at 5:35 PM says 'vprofile - #9 Started by user waseem akram (Open)'. The third message, at 8:20 PM, says 'hiring-app - #5 Started by user waseem akram (Open)' followed by 'hiring-app - #5 Success after 6 sec (Open)'. At the bottom is a text input field with a rich text editor toolbar containing icons for bold, italic, underline, link, unlink, bulleted list, numbered list, indent, code, and quote.

jenkins APP 4:40 PM
Slack/Jenkins plugin: you're all set on <http://0.0.0.0:8081/>
vprofile - #7 Started by user waseem akram ([Open](#))
vprofile - #7 Success after 12 sec ([Open](#))

jenkins APP 5:31 PM
vprofile - #8 Started by user waseem akram ([Open](#))
5:35 vprofile - #9 Started by user waseem akram ([Open](#))

jenkins APP 8:20 PM
hiring-app - #5 Started by user waseem akram ([Open](#))
hiring-app - #5 Success after 6 sec ([Open](#))

Conclusion:

—-> we are able too see from task 1 and task 2 both jobs are being sending alerts

Job1 from task1 : vprofile

Job2 from task2 : hiring-app

Error faced :

In step 5 of task2 I have given password as **sonar@123** but in step 9

I added password as **sonar123** so the sonarqube didn't recognise the password so I changed the password to **sonar123** and restarted postgresql and again ruined the sonarqube and after that the port runned

Step 5 : Login to PostgreSQL

```
sudo -u postgres psql
```

—>Create Database for SonarQube

Inside PostgreSQL prompt:

```
CREATE DATABASE sonarqube;
```

—->Create SonarQube User

```
CREATE USER sonar WITH ENCRYPTED PASSWORD 'sonar@123';
```

—> Grant Privileges

```
GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonar;
```

—>Exit from PostgreSQL

```
\q
```

Step 9 — Configure SonarQube Database Connection

```
sudo vi /opt/sonarqube/conf/sonar.properties
```

Find and update these lines:

```
sonar.jdbc.username=sonar  
sonar.jdbc.password=sonar123  
sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube
```



Also set **web port**:

```
sonar.web.host=0.0.0.0  
sonar.web.port=9000
```